Document number: P0341R0
Date: 2016-05-30
Reply-To: Mike Spertus, Symantec (mike_spertus@symantec.com)
Audience: SG7 Reflection possibly also Evolution Working Group

parameter packs outside of templates

Abstract

Parameter packs have proven exceedingly useful within templates but are unavailable in other contexts where they could prove equally useful. This paper addresses this by extending and generalizing the concept of parameter pack, and more generally of template argument lists, to a much wider range of contexts. Numerous use cases are given.

Proposal

We will build up our definitions incrementally from consideration of natural use cases.

Parameter pack declarations

We start by observing that even within a template class, the use of parameter packs is uncomfortably restricted:

```
template<typename ...T>
struct easy_tuple {
        template<typename U> easy_tuple(U &&u) : t{u}... {}
        T ...t;  // Ill-formed. Parameter pack members not allowed
};
```

The above code appears perfectly natural and leads to a far cleaner and easier to understand implementation of tuple than the usual one. Motivated by this, we propose removing the restriction that only parameter declarations can be packs.

Here is another useful example of non-parameter declarations of packs.

```
template<typename ...T> T cleanData(T t); // Performs data cleaning: normalizing, removing outliers, etc.
template<typename ...T> void processData(T ...rawData) {
        T ...cleanedData{cleanData(raw_data)...};
        /* Work with cleanedData */
};
```

Accessing a template parameter pack outside the template

Accessing the member packs from outside the template is useful as well, for example to implement `easy_tuple`s `get` accessor.

```
template <int i, typename ...T> auto ith_argument(T && ...t) { /* Usual code to return ith argument }

template<typename int i; typename ...T> auto get(easy_tuple<T...> tup) {
        return ith_argument<i>(tup....t& ...);  // Access parameter pack from outside its defining template
}
```

Note: By analogy with how we write `a.template b<int>` to indicate that a member is a template, the above code adopt as a bikeshed Richard Smith's suggestion to use `...` to indicate a member of a dependent type is a pack.

Pack literals

So far, our only way to create a parameter pack is via a template. This means that to create a "typelist" consisting of `int` and `double` would require detouring through the cumbersome `easy_tuple<int, double>::t`. To simplify the creation of packs, we propose angle bracket-enclosed template argument lists, such as `<int, double>` as pack literals. As an example, we can reduce the amount of "special cases" in the C++ standard by eliminating the following carve out from §14.1p9[temp.param] which says that " A default *template-argument* may be specified for any kind of *template-parameter* (type, non-type, template) ~~that is not a template parameter pack~~."

```
template<typename...T = <double, double> > // default to 2-dimensional double-valued Euclidean space
struct euclidean_space { /* ... */ };
```

Pack values

To the uninitiated, it may seem surprising that the following is ill-formed.

```
auto f() { return {2, "foo"}; // Ill-formed. braced-init-list is not an expression}
```

The code *looks like* it is returning an expression, and it *seems like* the expression should have a type. However, there is special

language in the standard for returning a *braced-init-list* because, although it may often be used like an expression, it is not one. With packs representing their type, *braced-init-lists* can be first-class expressions, again reducing the number of special cases:

```
auto f() { return {2, "foo"}; } // Now OK. Return type is <int, char const *>
```

As in the last section, we can also remove the restriction on packs having default arguments for for function parameters from §8.3.6p3 [dcl.fct.default].

```
template<typename ...T>
void recognize(T ...ourHeroes = {"Alan Turing", "Dennis Ritchie", "Bjarne Stroustrup"});
```

Of course, a parameter packs values can also be returned directly.

```
template<typename ...T>
auto truncate(T ...t) { return min(100, t)...; }
```

Returning a pack rather than a `std::pair` or a `std::tuple` eases composition and functional programming by increasing loose coupling.

```
<double, double> calculateTargetCoordinates();
double distanceFromMe(double x, double y);

void launch() {
        if(distanceFromMe(calculateTargetCoordinates()...))
                getOuttaHere();
}
```

Note: As mentioned above. Our packs generalize template parameter lists. All packs are types and have instantiable values (provided the types in the pack do). This is straightforward for type parameter packs. E.g., `{2, 'c'}` is a valid value of the pack type `<int, char>`. A more interesting example would be the type <string, 7, int> which is a valid template parameter list and therefore represents a pack type. A valid value for this type is `{"C++"S, 7, 5}`. By encoding known values in the pack type, (possibly significant) storage can be saved in templatized programs.

Named packs

Pack literals can also name their members, providing a very natural solution to the pack analogue of “named tuples” problem, making it easy to create functions that return multiple values descriptively.

```
<string topSong, person president, double avgTemp> someFactsAboutYear(int year) {
  if(year==1962)
    return {"Stranger On The Shore", Presidents.get("Kennedy"), 14};
}
```

Named packs are implicitly convertible to and from unnamed packs but not each other for added static type safety beyond what is provided in `pair` and `tuple`.

Typelists

Another use case we would like to cover with parameter packs is as typelists. For this and other reasons, we propose that packs are types and can be used as types without being expanded. We illustrate the notation for partially specializing a template on typelists with an `append` metafunction for concatenating two typelists.

```
template<typename L, typename R> struct append;

template<typename ...Ls, typename ...Rs>
struct append< <Ls...>, <Rs...> > {
        using type = <Ls..., Rs...>;
};
```

Note that we are not limited to types and can also do intlists, etc.

```
template<typename theoretical, typename actual> struct accuracy;

template<double... predictions, double... measurements>
struct accuracy< <predictions...>, <measurements...> >) { /* ... */ };
```

Using packs as a base typelist type also makes it easier to coexist with the myriad of existing and experimental typelist classes, such as those in Boost MPL, Loki, with more under development. For example, given a pack `T = <int, double, long>`, we can convert it to a `boost::mpl::vector` as simply as `mpl::vector<T...>`

Note that we do not make any claim that packs are the only way to do typelists, but merely that they co-exist well with and

between other approaches. See the discussion below about user-defined classes.

Reflection

We think packs are a reasonable choice for reflection typelists. For example, it naturally implements all of the motivating examples from N2965, which benefit from C++ pack expansion of base classes. Packs also lend themselves to reflection facilities like allowing a function to get a pack of all of its parameters or a pack of all of the fields of a class. At the same time, other, perhaps richer, models of typelists could well be considered by SG7 as an alternative to the low-level packs used here (Of course, that would not be an argument against any of the pack enhancements in this paper).

Iterator interface

Like `boost::MPL`, parameter packs have iterator interfaces. They have `pack_begin` and `pack_end` type traits. E.g., `pack_begin_t< <int, double> >` gives an "iterator" that has `current` and `next` members. It also has a `get` member that accepts a value for that type and returns the corresponding element. In the above case, `get<pack_begin_t< <int, double> > >({3, 4.5})` returns `3`. Indexed versions should also be available. Together, this allows iteration and Boost::Fusion-style processing of packs.

User-defined types

C++ strives for user-defined constructs are generally designed to be as close to built-in constructs as possible. While providing a pack interface to user-defined types is separable from the pack deserves a separate proposal, we provide a brief sketch here with significant inspiration from Richard Smith's ideas on `operator...()`.

We consider `tuple` as a prototype. For example, we would want `tuple<int, double>...` to be the pack `<int, double>` and `tuple<int, double>{2, 3.5}...` to be the `{2, 3.5}` value of the pack type `<int &, double &>`. Here is how these could be implemented in the `easy_tuple` example above.

```
template<typename ...T>
struct easy_tuple {
        /* ... */
        <T&...> operator...() { return t...; }
};

template<typename ...T>
using easy_tuple<T...>... = <T...>;
```

Notes:

- This makes the much-desired feature of simply expanding a tuple into the arguments of a function trivial: `f(...t...);`. It is instructive to look at DRayX's long and complexly metaprogrammed C++14 workaround for this feature by comparison.
- As Richard Smith has pointed out, we want the `...t...` above to "pack-ize" the `t` because otherwise any pack expression containing a tuple would suddenly unpack the tuple, breaking compatibility, and likely not what would be desired regardless.
- By default, it `T` is a user-defined type that defines an `operator...`, then `...T...` will be `<...declval<T>()...>`. However, it is possible, and maybe desirable as illustrated by the `easy_tuple` code, to override that.
- If a class defines an `pack_begin` iterator interface, then pack expansion is automatically generated without need to write an `operator...()`.
- For some types, like intlists, `...` should be able to produce a value rather than just a type, just like with non-type template pack parameters. In this case, a static `operator...` method may be supplied.